# Rapport de conception - Projet Client-Serveur

Mathis HURSTEL, SEOC Phelma

November 29, 2024

## Contents

**1 Introduction** **2**

**2 Architecture du code** **2**
2.1 Modularité du projet . . . . . 2
2.2 Gestion des processus : Lecteur et Rédacteur . . . . . 3
2.3 Choix entre `thread` et `fork` . . . . . 3

**3 Outils de commnication** **4**
3.1 Création d'un tampon local . . . . . 4
3.2 Utilisation des sémaphores . . . . . 5
3.3 Utilisation des pipes . . . . . 5

**4 Gestion des erreurs** **5**

**5 Difficultés rencontrées et solutions adoptées** **6**
5.1 Synchronisation des processus . . . . . 6
5.2 Appréhension des concepts . . . . . 7
5.3 Ecriture du Driver . . . . . 7

**6 Bilan du projet** **7**
6.1 Conclusion . . . . . 7
6.2 Critiques et pistes d'amélioration . . . . . 8

# 1 Introduction

Ce rapport présente les choix stratégiques effectués pour la conception du programme client dans le cadre du projet Client-Serveur. Il expose la structuration du projet, les mécanismes de gestion des processus et des communications, ainsi que les défis rencontrés. Ce projet vise à permettre au client de se connecter au serveur, de recevoir des données issues de plusieurs voies et de les transmettre à un module d'affichage en respectant les contraintes définies.

# 2 Architecture du code

## 2.1 Modularité du projet

Le projet a été organisé en plusieurs fichiers source distincts pour garantir une conception modulaire et réutilisable. Cette méthodologie permet de clarifier un projet à la structure complexe dûe à l'exécution indépendante et simultanée de multiples clients et processus.

- **CL.c (main)** : Point d'entrée principal pour le programme client. Il fait office de main() et initialise les composants afin de superviser l'exécution globale des clients. Il est responsable de la bonne synchronisation entre les processus afin de lire et écrire les données en empêchant toute interférence entre ces dernières. Il a pour responsabilité la création des processus, des pipes et des sémaphores et donc également leur fermeture afin de garantir aucune fuite mémoire.

- **moniteur.c** : Responsable de la gestion des connexions du client au serveur, de la création des processus et de la synchronisation des données. Il offre une porte d'accès à la file de messagerie publique dans un premier temps, puis au serveur dans le but de rejoindre la mémoire partagée. Ces autorisations se font via un échange de messages contenant des chemins d'accès, créant elles-mêmes des clé d'accès et des identifiants.

- **lecteur.c** : Contient les fonctions permettant de lire les données dans les tampons partagés. Nous avons fait le choix de créer un processus par voie afin d'empêcher les données de se mélanger et de se perdre dans les pipes et tampons. Ce choix de conception simplifie grandement la communication inter-processus mais impose certaines limites sur lesquelles nous reviendrons par la suite.

- **rédacteur.c** : Traite les données lues par les lecteurs et les transmet au *Driver*. Là encore, un processus a été affecté à chaque voie, et donc par extension à chaque lecteur. On utilise donc un sémaphore pour ordonner l'accès au Driver.

- **driver.c** : Gère l'affichage des données sur la console. Il récupère les données par paquet de 5 afin de fluidifier les échanges Serveur-Client. Son implémentation se fait en dernière étape de par sa complexité et son indépendance relative comparée à l'imbrication mutuelle de toutes les autres fonctions.

- **sem.c** : Implémente les fonctions nécessaires pour gérer les sémaphores utilisés dans la synchronisation. On y a recours pour alléger les fonctions du main (CL.c) déjà très complet du fait de sa gestion des processus et des cas d'erreur.

Cette organisation facilite la maintenance et le débogage du programme, permet des tests isolés de chaque composant et rend le projet facilement extensible. Grâce à la modularité des fonctions, la compilation peut aisément se faire de manière progressive afin d'améliorer l'efficacité du code ou régler un problème.

## 2.2 Gestion des processus : Lecteur et Rédacteur

Le processus **Moniteur** crée quatre sous-processus : deux Lecteurs (Lecteur1 et Lecteur2) et deux Rédacteurs (Rédacteur1 et Rédacteur2). Chaque couple Lecteur-Rédacteur est associé à une voie spécifique. Voici le rôle de chaque composant :

- **Lecteur** : Lit les données depuis la mémoire partagée du serveur en utilisant un tampon local associé, tout en garantissant l'exclusion mutuelle grâce aux sémaphores.
- **Rédacteur** : Accumule les données lues par le Lecteur et les transmet par blocs au *Driver* via des pipes pour optimiser les échanges.
- **Moniteur** : Supervise les Lecteurs et Rédacteurs, reçoit les signaux du serveur, et gère les communications entre les processus.

Cette gestion multi-processus garantit une séparation claire des responsabilités, tout en augmentant la complexité du système de synchronisation.

## 2.3 Choix entre `thread` et `fork`

Dans la conception d'un programme complexe comme celui du projet Client-Serveur, un des premiers choix architecturaux est de déterminer comment gérer l'exécution concurrente des différentes tâches. Les deux principales options sont :

1. `thread` : plusieurs fils d'exécution au sein d'un même processus, partageant son espace mémoire.
2. `fork` : plusieurs processus fils, chacun ayant son propre espace mémoire indépendant.

Nous avons opté pour l'utilisation de `fork` dans ce projet. Voici une analyse détaillée des deux approches et des raisons de notre choix.

#### 1. `thread`

Un `thread` (fil d'exécution) est une unité légère qui partage l'espace mémoire et les ressources de son processus parent. Plusieurs threads s'exécutent de manière concurrente dans un même processus, partageant la mémoire globale et les ressources, ce qui permet une communication directe entre eux.

Avantages : **Rapidité** : La création d'un thread est plus rapide que celle d'un processus, car il ne duplique pas l'espace mémoire. **Communication simplifiée** : Les threads partagent la mémoire globale, ce qui facilite l'échange de données. **Faible coût en ressources** : Les threads consomment moins de mémoire que des processus séparés.

Inconvénients : **Complexité de synchronisation** : Le partage de mémoire nécessite une gestion stricte des accès (mutex, sémaphores) pour éviter les erreurs. **Vulnérabilité** : Une erreur dans un thread peut affecter l'ensemble du processus.

**2.** `fork`

La fonction `fork` crée un nouveau processus fils, qui possède son propre espace mémoire séparé du parent. Les modifications dans le fils n'affectent pas le parent, et la communication entre processus nécessite des mécanismes explicites comme les pipes ou la mémoire partagée.

Avantages : `fork` permet l'isolation des processus, garantissant qu'une erreur dans un processus n'affecte pas les autres, ainsi qu'une certaine robustesse, puisque chaque processus est indépendant, ce qui facilite le débogage. De plus, il n'y a pas de synchronisation mémoire nécessaire, car les processus n'ont pas de mémoire partagée par défaut.

Inconvénients : En revanche, l'utilisation de `fork` présente un coût plus élevé en temps et en mémoire comparé à `thread`, ainsi qu'une communication plus complexe, nécessitant des mécanismes comme les pipes ou la mémoire partagée pour échanger des données.

**3. Résumé du choix**

| **Critères** | **Threads** | **Fork** |
|---|---|---|
| **Performances** | Création rapide | Plus lourd en ressources |
| **Communication** | Mémoire partagée, simple | Pipes/mémoire partagée, plus complexe |
| **Robustesse** | Faible (partage mémoire) | Forte (isolation des processus) |
| **Synchronisation** | Mutex/sémaphores nécessaires | Pas de synchronisation mémoire |
| **Débogage** | Difficile (partage mémoire) | Plus simple (processus isolés) |

Table 1: Comparaison entre `thread` et `fork`

# 3 Outils de commnication

## 3.1 Création d'un tampon local

Un tampon local est créé pour chaque Lecteur, permettant de récupérer les données issues de la mémoire partagée du serveur. Ce tampon est une structure liée à la mémoire partagée et suit une organisation stricte :

- Les données sont stockées de manière binaire : les indices pairs (`n`) correspondent à la voie 1, les indices impairs (`n+1`) à la voie 2. Cela simplifie l'accès par pointeur et garantit que chaque Lecteur récupère uniquement les données qui lui sont destinées.
- Les tampons sont des **buffers circulaires**, ce qui évite de gérer dynamiquement leur taille. Lorsqu'un tampon est plein, l'écriture recommence au début, écrasant les données les plus anciennes (principe FIFO).

Cette structure garantit une gestion optimisée de la mémoire et une récupération ordonnée des données par les Lecteurs.

### 3.2 Utilisation des sémaphores

Les sémaphores jouent un rôle central pour sécuriser les accès concurrents :

- **Entre les Lecteurs et le tampon partagé** : Chaque Lecteur doit attendre que le sémaphore associé à la voie soit libéré avant de lire ou écrire. Cela empêche des Lecteurs de se "voler" des données ou d'écrire en dehors de leur section dédiée.
- **Entre les Rédacteurs et le *Driver*** : Les Rédacteurs doivent accéder au *Driver* par lots (ici 5 données) tout en s'assurant qu'un seul rédacteur écrit à la fois. Les sémaphores locaux garantissent cet ordre.

Deux niveaux de sémaphores sont utilisés :

1. **Premier sémaphore** (géré par le serveur) : Garantit la gestion ordonnée des tampons partagés.
2. **Deuxième sémaphore** (local au client) : Synchronise les interactions entre Lecteurs et Rédacteurs pour éviter la corruption des données.

L'implémentation des sémaphores dans le `main` centralise leur gestion, facilitant ainsi le débogage.

### 3.3 Utilisation des pipes

Les pipes permettent une communication directe et simplifiée entre :

- Lecteurs → Rédacteurs : Transmission des données issues des tampons.
- Rédacteurs → *Driver* : Envoi des lots de données pour affichage.

Ce choix offre une gestion unifiée, mais pose certaines limites :

- Les pipes peuvent être remplis, imposant une gestion stricte de la taille.
- Les pipes doivent être correctement fermés (`close`) et libérés (`exit`) pour éviter les fuites de ressources.
- Contrairement à un tampon client local, un pipe est limité en termes de stockage, ce qui impose une discipline stricte dans le traitement des données.

La simplicité d'utilisation des pipes a été un facteur clé dans ce choix.

## 4 Gestion des erreurs

Les erreurs sont classées en plusieurs catégories :

- **Client (`CLEerr`)** : Échec de connexion ou déconnexion due à une saturation du serveur.
- **Sémaphores (`SEMerr`)** : Échec d'initialisation ou de synchronisation.
- **Mémoire partagée (`SHMerr`)** : Échec d'accès à la mémoire partagée.

- **Messages (`MSGerr`)** : Problèmes lors de l'envoi ou de la réception.

Chaque fonction retourne un code : - **0** : Succès. - **<0** : Erreur, avec un code spécifique à la nature du problème. - **1** : Comportement géré mais non souhaité (par exemple, client déconnecté faute de place).

Nous avons notamment mis un point d'honneur à évoluer de la manière la plus rigoureuse possible tout au long de ce projet. Ainsi, la création de chaque structure ou l'appel à n'importe quelle fonction est accompagnée d'un traitement des erreurs éventuelles. On compte parmi celles-ci l'incompatibilité des paramètres fournis en entrée, un retour de fonction différent du résultat escompté ou bien un processus incapable de réaliser la tâche demandée (allocation d'espace, création d'une clé d'accès...).

# 5 Difficultés rencontrées et solutions adoptées

## 5.1 Synchronisation des processus

Assurer une synchronisation parfaite entre Lecteurs et Rédacteurs a nécessité de nombreux ajustements car il s'est avéré bien plus compliqué que prévu d'appeler les bonnes fonctions au bon moment afin de ne râter aucune donnée. L'organisation entre le main, les processus, les fichiers auxiliaires et les fonctions litbuf() a nécessité beaucoup de réflexion sur l'architecture du programme.

### Gestion de la temporisation des lectures de données

Pour garantir que les Lecteurs accèdent aux données écrites par le serveur au bon moment, plusieurs stratégies ont été envisagées :

- `sleep()` : Cette méthode suspend temporairement l'exécution du processus pour une durée prédéterminée. Elle est simple à implémenter mais repose sur une estimation arbitraire de la durée nécessaire, ce qui peut entraîner des attentes inutiles ou, à l'inverse, des lectures prématurées.

- `pause()` : Le processus reste suspendu jusqu'à la réception d'un signal, garantissant une reprise immédiate dès que les données sont prêtes. Cependant, cette méthode exige une gestion rigoureuse des signaux (comme `SIGUSR1` ou `SIGUSR2`), avec le risque qu'un signal perdu bloque le processus indéfiniment.

- **Masques de signaux** : En utilisant des fonctions comme `sigemptyset`, `sigaddset`, et `sigprocmask`, il est possible de bloquer temporairement certains signaux pendant l'exécution d'une section critique, tout en les laissant en attente pour traitement ultérieur. Cela offre un contrôle précis, mais complexifie le développement.

Dans ce projet, `sleep()` a été privilégié pour sa simplicité et son indépendance vis-à-vis des signaux. Bien que cette approche introduise un délai fixe qui peut ne pas être optimal, elle garantit un fonctionnement robuste dans un contexte à faible charge. Pour sécuriser certaines sections critiques, des masques de signaux (`sigprocmask`) ont également été utilisés pour éviter toute interférence. Une future amélioration pourrait combiner `pause()` avec une gestion avancée des signaux afin d'augmenter la réactivité et réduire les temps d'attente inutiles.

## 5.2 Appréhension des concepts

Ce projet a permis d'aborder de nouveaux concepts que nous n'avions pour la plupart pas encore eu l'occasion de traiter. Ainsi, bien que ce n'est certainement pas un projet particulièrement volumineux, il était assez fastidieux de se lancer dans la programmation sans avoir parfaitement assimilé les notions du cours.

Il a donc été nécessaire de bien travailler en amont sur l'architecture du programme, l'imbrication des processus et le contenu des fonctions, afin d'éviter à tout prix de se jeter à l'aveugle dans du codage instinctif. Une connaissance précise des directions à suivre pour évoluer efficacement permettait de ne pas s'éparpiller dans tous les sens en programmant plusieurs fonctions/processus à la fois.

## 5.3 Ecriture du Driver

Le module *Driver* a nécessité plus de temps qu'il n'en parait nécessaire et ceci est dû ) plusieurs raisons. Bien que clair en apparence, son rôle au sein du projet s'avère plus complexe qu'en surface car il intervient à plusieurs niveaux (rédacteur, sémaphore et pipe). Étant la cerise sur le gâteau du projet, il permettait sans trop le vouloir de mettre en avant les défauts de conception du gâteau en question. Une mauvaise structure, bien que minime (paramètres, retour, endroit d'implémentation d'une fonction.) demandaient à revoir plusieurs parties du programme car elles empêchaient la bonne exécution des parties finales.

Ainsi, malgré un bon travail en amont, le Driver et son débogage dans la compilation du client s'est avéré plus fastidieux et chronophage que prévu. Il a donc été nécessaire d'utiliser sans retenu des printf() à diverses endroits du code Client ET Serveur afin de repérer les lignes exécutées ou non. Ce passage en revu minutieux permettait néanmoins de clarifier le code et d'en comprendre réellement la structure interne

# 6 Bilan du projet

## 6.1 Conclusion

Le projet a permis de traiter les contraintes d'une communication Client-Serveur en temps réel, en implémentant les structures nécessaires à la bonne transmission réciproque des données. Il permettait d'aborder avec une grande autonomie le fonctionnement interne d'un système semi-complexes de processus.

Les contraintes ont été respectées puisque c'est le moniteur qui appelle ses 4 processus fils, et que chaque lecture/écriture de donnée est précédée d'un appel au sémaphore pour assurer l'intégrité des messages. Le programme est modulé dans différents fichiers aux fonctions et buts distincts, les lecteurs sont associés respectivement au rédacteur de la voie correspondante et tout l'affichage final est géré par le processus Driver.

La programmation du projet s'est évidemment faite de manière propre et commentée en gérant les différentes erreurs envisageables et en offrant des pistes de compréhension à un observateur extérieur.

## 6.2 Critiques et pistes d'amélioration

Le code est sécurisé mais volumineux et peu optimisé. Les cas d'erreur sont appréhendés, au détriment de la lisibilité et de l'efficacité du code. Cela entraîne un alourdissement quelques fois non nécessaire des fonctions et une répétition de certaines lignes/vérifications.

Réduire sa taille ou explorer des solutions comme des tampons plus flexibles pourrait l'améliorer de manière significative. Les structures de données et les méthodologies de conception adoptées ont également été influencées par ce soucis de favoriser la simplicité du projet. Nous avons implémenté des processus volontairement gourmands mais fiables et modulables.